NEXORA



Predictive Maintenance

and Machine Failure Prediction System

PROJECT PROPOSAL REPORT

Phase 1: ETL + EDA Pipeline | Programming for AI | Updated proposal (Streamlit edition)

GROUP

NEXORA

GROUP MEMBERS

Areeba Sattar — 2025-AI-124

Parishay Ajaz — 2025-AI-133

Muhammad Awais Raza — 2025-AI-141

Ghulam Mujahid — 2025-AI-151

SUBMITTED TO

Mr. Shaban Satti

SECTION

Section C — Artificial Intelligence

01 INTRODUCTION

Course: Programming for AI · **Project:** Predictive Maintenance and Machine Failure Prediction System. **Phase:** 1 (CLO2)

1.1 Executive Summary

Unscheduled machine downtime causes heavy financial losses, safety risks and logistical bottlenecks in industrial plants. This project builds an end-to-end system that turns raw industrial sensor telemetry into validated data, insight and a failure-risk estimate. The proposal has been updated to a **real industrial dataset**, the Kaggle *Pump Sensor Data* file (**sensor.csv**), and to a **Streamlit** web application that runs the whole pipeline and presents the results. Phase 1 delivers the data foundation (ETL and EDA) for the machine-learning and deep-learning phases that follow.

1.2 Phase-1 Objectives

- An automated **ETL (Extract, Transform, Load)** pipeline that reads the pump sensor file from the project backend and runs on every app start.
- A strict **data-quality and validation framework** (pandas, NumPy, Pydantic v2) in which invalid rows are quarantined, never silently lost.
- A persistent storage layer: a processed **CSV** and a **SQLite** database of the cleaned, feature-engineered data.
- Structured **EDA** with Matplotlib, with before / after cleaning comparisons for all sensors.
- An interactive **Streamlit dashboard**, deployed online, with a baseline failure-risk inference page.

02 PROBLEM STATEMENT & MOTIVATION

Industrial maintenance follows two legacy paradigms, both flawed. **Reactive maintenance** (run-to-failure) lets pumps break down, causing secondary damage, safety hazards and long stoppages. **Preventive maintenance** (time-based) replaces components on fixed schedules, discarding parts with useful life left and still missing failures that happen between inspections.

A second problem lies in the data. The pump dataset reflects what real telemetry looks like: **52 anonymous sensors** read once per minute, one sensor that is completely empty, another that is missing about a third of its readings, long stretches of missing values, and a machine state that is almost always normal. Only **7 breakdowns** occur in 220,320 records, and each is followed by thousands of "recovering" records whose sensor values look nothing like normal operation. A model trained directly on this raw data would either have almost nothing to learn from or would learn the recovery phase instead of the warning signs. A repeatable, validated pipeline and a carefully defined prediction target are needed first.

PROPOSED STRATEGIC SOLUTION

Condition-based, predictive maintenance. The system cleans and validates the multi-sensor stream, measures how far every reading drifts from healthy behaviour, and estimates the probability that the pump will break **within the next 24 hours**. Phase 1 delivers the trustworthy data, the features and a working baseline that Phases 2 and 3 build on.

03 DATASET: KAGGLE PUMP SENSOR DATA (SENSOR.CSV)

The data comes from Kaggle (*nphantawee / pump-sensor-data*). The CSV is stored in the project backend at `data/raw/sensor.csv` and is read automatically; the user never uploads it. If the file is missing, the app can fetch it once with *kagglehub*.

Property	Description
Source / file	Kaggle *Pump Sensor Data*, single file **sensor.csv** (CSV, about 120 MB).
Size and period	220,320 records, one reading per minute, April to August 2018, from one industrial pump.
Columns	`timestamp`, 52 sensor channels `sensor_00` … `sensor_51` (anonymous names and units), `machine_status` (plus an unnamed index column, removed on load).
Machine status	**NORMAL** (about 205,800 rows), **RECOVERING** (about 14,500 rows), **BROKEN** (7 rows).
Known data issues	`sensor_15` is 100% empty; `sensor_50` is missing about 35% and `sensor_51` about 7%; smaller gaps in several others; sensors flat-line or drop to zero around failures.
Target variable	Not in the file. Built as **breakdown_flag** = 1 if a BROKEN event occurs within the next 24 h (about 4–5% of rows, similar to the imbalance expected in the original plan).

04 SYSTEM ARCHITECTURE & APPROACH

The system is organised as five sequential layers inside one Python code base, from reading the backend CSV to presenting analytics in Streamlit. It replaces the earlier plan of a separate Flask REST service with an HTML/CSS/JavaScript front end: the real dataset is a static file, and a single Streamlit application keeps backend and front end together and is simple to deploy.

Layer	Responsibility
1 · Extraction	Reads **sensor.csv** from the backend, removes the index column, checks that all 54 expected columns exist; optional *kagglehub* download.
2 · Cleaning & Validation	pandas / NumPy cleaning, then a Pydantic v2 contract on every row; failing rows go to a quarantine table with the reason.
3 · Features & Target	Healthy-baseline z-scores, deviation features, rolling statistics and the 24-hour early-warning label.
4 · Persistence	Processed CSV (`data/processed`) and SQLite database (`data/maintenance.db`) written on demand.
5 · Analytics & Presentation	Matplotlib EDA and a baseline risk model, shown in a four-page Streamlit dashboard.

05 TECHNICAL STRATEGY

5.1 Extract — Telemetry Ingestion

What: loading the 220,320-row pump file with pandas. **How:** a path configured once in `config.py`; the file lives in the backend, so the pipeline starts automatically when Streamlit starts. **Why:** a stable, reproducible source; for demonstration, an optional *noise injector* corrupts a copy of the data (dropouts, spikes, −999 codes, messy status text, duplicate rows) so the before / after cleaning plots show real work.

5.2 Transform & Validate — Data Quality Assurance

- **Structure:** drop the empty `sensor_15`; remove duplicate rows and duplicate timestamps; parse and sort by time; standardise *machine_status* (trim, upper-case, reject unknown values).
- **Invalid values:** sentinel codes (−999) and infinities become missing; optional per-sensor limits can be added once units are known.
- **Outliers:** an IQR fence (Q3 + 8 × IQR) computed and applied on NORMAL rows only, so the extreme readings of BROKEN and RECOVERING rows are kept. Naturally heavy-tailed sensors are left untouched.
- **Imputation (time-series aware):** gaps up to 30 minutes are linearly interpolated; longer gaps use the median of the same machine status; the global median is the last fallback.
- **Pydantic v2 contract:** valid timestamp, status in {NORMAL, BROKEN, RECOVERING}, target 0/1 and 51 finite sensor values. Bad rows are quarantined with the reason and counted in the report.

5.3 Feature Engineering and Target

- **Healthy-baseline z-scores:** every sensor is expressed in standard deviations from its mean during NORMAL operation (one z-column per sensor), which makes 51 differently scaled sensors comparable.
- **Deviation features:** mean and maximum |z| over all sensors, number of sensors beyond 3σ, and number of sensors reading exactly 0 (flat-line indicator).
- **Rolling features:** trailing 60-minute mean and standard deviation of the deviation score, capturing slow degradation.
- **Target:** because only 7 BROKEN rows exist, `breakdown_flag` marks the 24 hours before each breakdown (switchable to "abnormal state" in the config). `machine_status` is never a model input (target leakage).

5.4 Load — Persistence

The cleaned and engineered table is stored as a processed CSV and in a SQLite table (`pump_sensor_data`) through pandas and sqlite3; quarantined rows are listed with their rejection reason in the dashboard report. SQLite can be replaced by PostgreSQL without changing the pipeline.

5.5 Exploratory Data Analysis and Baseline Model

EDA covers missing-value analysis, distributions and boxplots before and after cleaning, a correlation matrix of the sensors most linked to failure, class balance, status distribution, telemetry trends around each breakdown, and raw-versus-clean signal streams. A fast **HistGradientBoosting** baseline powers the dashboard's Risk Inference page. With only 7 failures the train / test split is **by failure event** (the last events form the test set, no shuffling) and is judged by **PR-AUC and ROC-AUC**, not accuracy. Full modelling follows in Phases 2 and 3.

06 FRONTEND: STREAMLIT DASHBOARD

The front end is a single **Streamlit** app (`app.py`) with a maroon brand theme, light and dark mode, KPI cards and pill navigation. The pipeline result is cached, so clicks stay fast, and the sidebar offers a *Re-run pipeline* button and the noise-demo controls.

Page	Content
Dashboard	KPI cards (records, quarantined rows, failure-window rate, breakdown events, active sensors); sensor telemetry zoomable around any failure; failure records by status or time unit; raw-vs-validated stream; correlation of the sensors most linked to failure.
Before vs After	Missing values, distributions, boxplots, single-sensor comparison, sensor stream and a summary table, with counts of rows removed, duplicates, outliers and values imputed.
Risk Inference	Pick a historical reading (healthy baseline or 2 h before a failure), adjust the 12 most relevant sensors, and get a failure probability with a colour-coded verdict; model metrics shown in an expander.
Data & Report	Cleaning report (table, JSON, download), preview of the engineered data, CSV sample download, save to CSV / SQLite, and the quarantined rows.

07 DEPLOYMENT (STREAMLIT)

- **Platform:** Streamlit Community Cloud, deployed directly from the project's GitHub repository (main file `app.py`, dependencies in `requirements.txt`). Local run: `streamlit run app.py`.
- **Dataset in the backend: sensor.csv** (about 120 MB) exceeds GitHub's 100 MB per-file limit, so it is added with **Git LFS** or fetched at first start with *kagglehub* (Kaggle credentials stored in Streamlit secrets). It is never uploaded by the user.
- **Resources:** the pipeline result is cached in memory and the z-score columns use float32; if the free tier runs short, the app is demonstrated locally.

08 TECHNOLOGY STACK

Category	Chosen tool	Role
Language	Python 3.11+	Data engineering and machine-learning standard.
Frontend + backend	Streamlit	Interactive dashboard and pipeline runner in one app (replaces Flask + HTML/CSS/JS).
Validation	Pydantic v2	Row-level data contract; quarantine of invalid records.
Data processing	pandas, NumPy	Time-series cleaning, imputation and feature derivation.
Visualisation	Matplotlib	Before / after plots, telemetry, correlation, class balance.
Modelling	scikit-learn	HistGradientBoosting baseline (full models in Phase 2; PyTorch + MLflow in Phase 3).
Storage	CSV, SQLite	Processed data and database (PostgreSQL possible).
Data source / VCS	Kaggle, GitHub	sensor.csv dataset; source control and Streamlit deployment.

09 EXECUTION PHASES & RISKS

Phase	Tasks
1 · Foundation & Pipeline	Dataset selection (Kaggle Pump Sensor Data) — completed; config, extraction and cleaning modules; Pydantic schema; GitHub repository.
2 · Features, Storage & EDA	Feature engineering and target; SQLite / CSV load; before / after EDA; baseline model.
3 · Dashboard, Deployment & Viva	Streamlit app, deployment to Streamlit Cloud, README and setup guide, group viva preparation.

Risk	Impact	Mitigation
Only 7 failures; extreme class imbalance	High	24-hour early-warning label, event-based split, PR-AUC, class weighting / resampling in Phase 2.
Over-cleaning removes real failure signals	High	Outlier rules only on NORMAL rows; BROKEN / RECOVERING rows untouched.
Large file and memory limits when deployed	Medium	Git LFS or kagglehub; caching; float32; local demonstration as fallback.
Anonymous sensors limit physical rules	Medium	Statistical z-score features; per-sensor limits can be added later.
Time and scope pressure	Medium	Modular milestones; dashboard kept simple.

10 EXPECTED OUTCOMES

A validated ETL pipeline with pump data stored as CSV and SQLite, visual insight into sensor behaviour before failures, and a deployed Streamlit dashboard with a clean dataset ready for Phase 2 (scikit-learn) and Phase 3 (PyTorch + MLflow). *This report presents the plan and methodology; no experimental results are reported.*